**Core Functionality**

This document explains the main functions implemented for the Week 2 milestone of the Process Change Impact Simulator. The application is built with Python, Flask, SQLite, and NetworkX. The work completed so far focuses on creating process models, adding activities and dependencies, importing process data from JSON, checking process data, and finding activities affected by a change to an activity.

## 1. Selected Functional Requirements

The following requirements were selected from the project's original list of 50 functional requirements. Their original identifiers are kept below.

| Requirement | Original requirement | Functionality implemented |
|---|---|---|
| FR-01 | Process analysts can create a new business process by entering its name and description. | A process analyst can send a process name and description to the application. The application checks that a name is supplied and saves the process in the SQLite database. The API returns the saved process, including its ID. |
| FR-02 | Process analysts can view a list of saved business processes. | The application provides an endpoint that returns the saved processes. This operation is included in the automated test suite. |
| FR-05 | Process analysts can add a new activity to a process. | A process analyst can add an activity to an existing process. The activity includes a name and type. The supported types are task, decision, approval, and output. The application checks that the process exists and that the activity details are valid. |
| FR-09 | Process analysts can connect two activities by creating a dependency. | A process analyst can connect two activities using their IDs. The application checks that both activities belong to the selected process and checks the proposed graph before saving the dependency. |
| FR-13 | Process analysts can import a process definition from a supported JSON file. | The application accepts a JSON process definition through its import API. The JSON includes process details, activities, and dependencies. A sample file is included in examples/sample_process.json. |

| | | |
|---|---|---|
| FR-14 | System validation checks imported files before saving process data. | The importer checks the JSON structure and its required information before it creates database records. This prevents an invalid document from being imported. |
| FR-15 | System validation displays an error message when an imported file contains missing or invalid fields. | When required information is missing or a field has an invalid value, the import endpoint returns an error response with a message explaining the problem. |
| FR-17 | System validation checks whether all dependencies refer to existing activities. | During JSON import, dependency references are checked against the activity keys in the same document. The dependency API also checks that both activity IDs belong to the selected process. |
| FR-18 | System validation detects circular dependencies and displays a warning. | The application checks the directed graph for cycles. JSON imports containing a cycle are rejected. When a new dependency is added through the API, the candidate graph is checked before the dependency is saved. |
| FR-26 | System graph processing represents process activities and dependencies as a directed graph. | The graph component uses NetworkX to create a directed graph. Activities are represented as nodes, and dependencies are represented as directed edges. |
| FR-27 | System analysis identifies activities directly connected to a selected activity. | The impact analysis returns the activities that are immediately connected after the selected activity in the direction of the process flow. |
| FR-28 | System analysis identifies all reachable downstream activities affected by a selected change. | The graph analysis finds all activities that can be reached downstream from the selected activity, not only the next activity. |
| FR-30 | System analysis distinguishes directly connected activities from indirectly connected activities. | The result separates immediate downstream activities from activities that are reached through one or more intermediate activities. |

| | | |
|---|---|---|
| FR-32 | Process analysts can view identified affected activities in a readable list. | The impact endpoint returns the changed activity, direct downstream activities, indirect downstream activities, and all affected activities as structured JSON. |

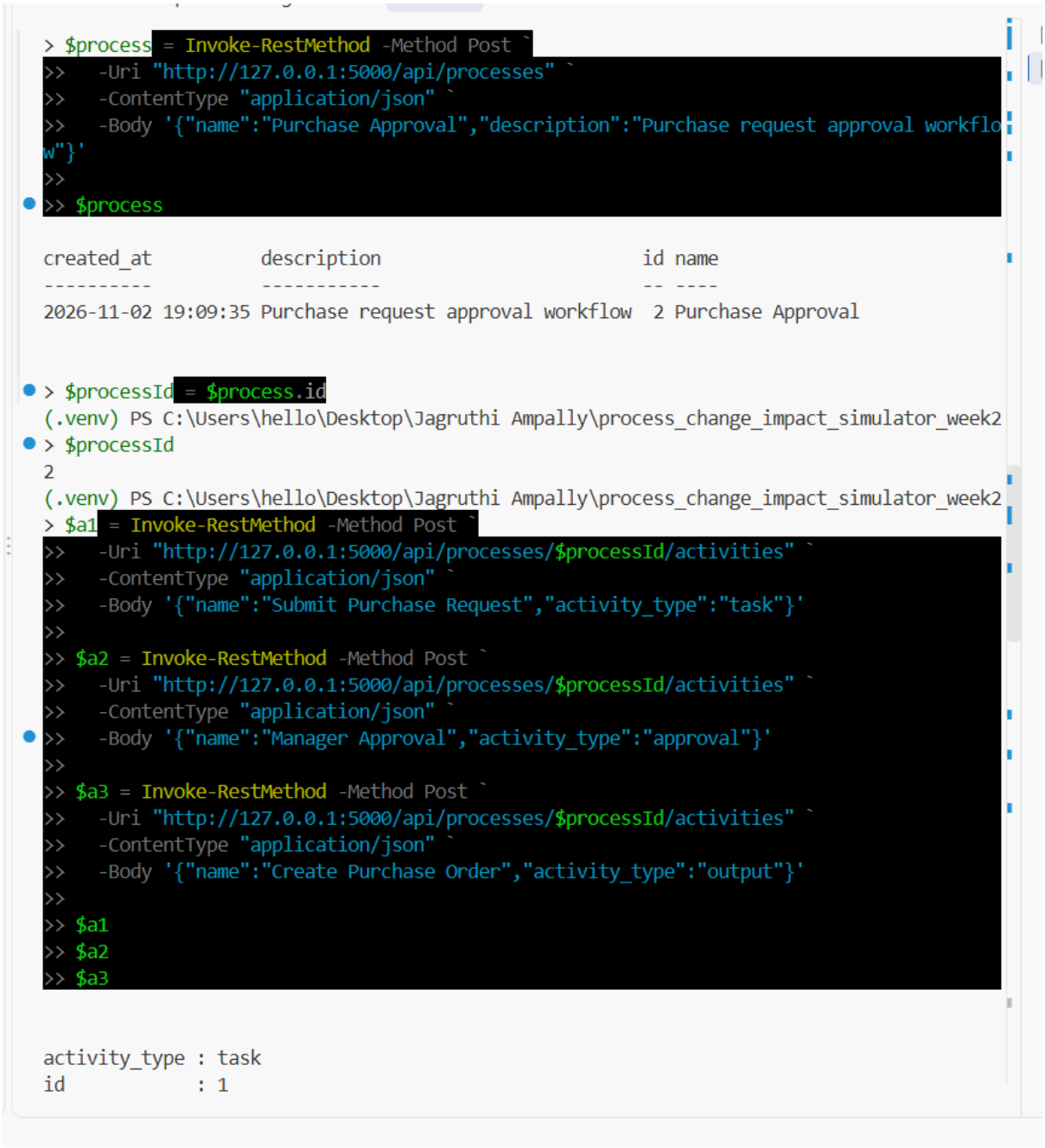
## 2. Source Modules and Components

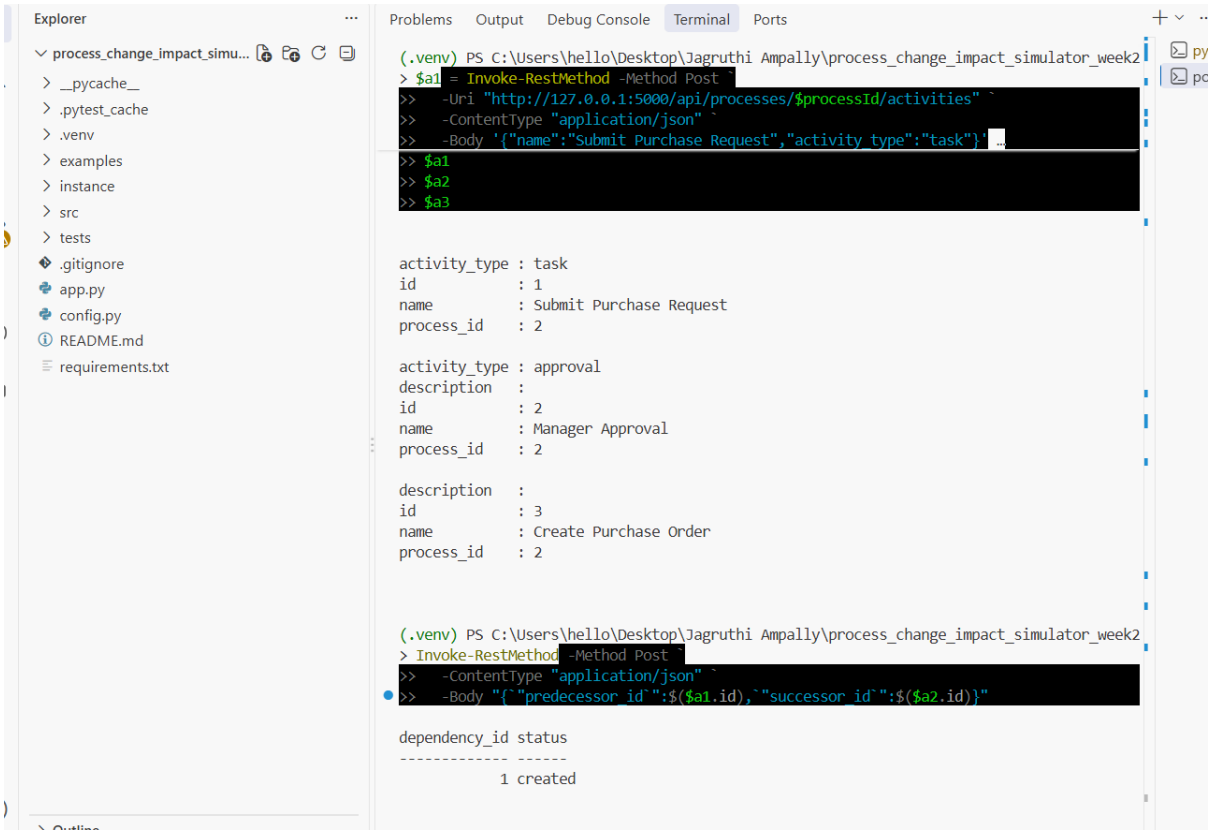
The application code is divided into smaller modules so that database work, process logic, graph analysis, and API requests are kept separate.

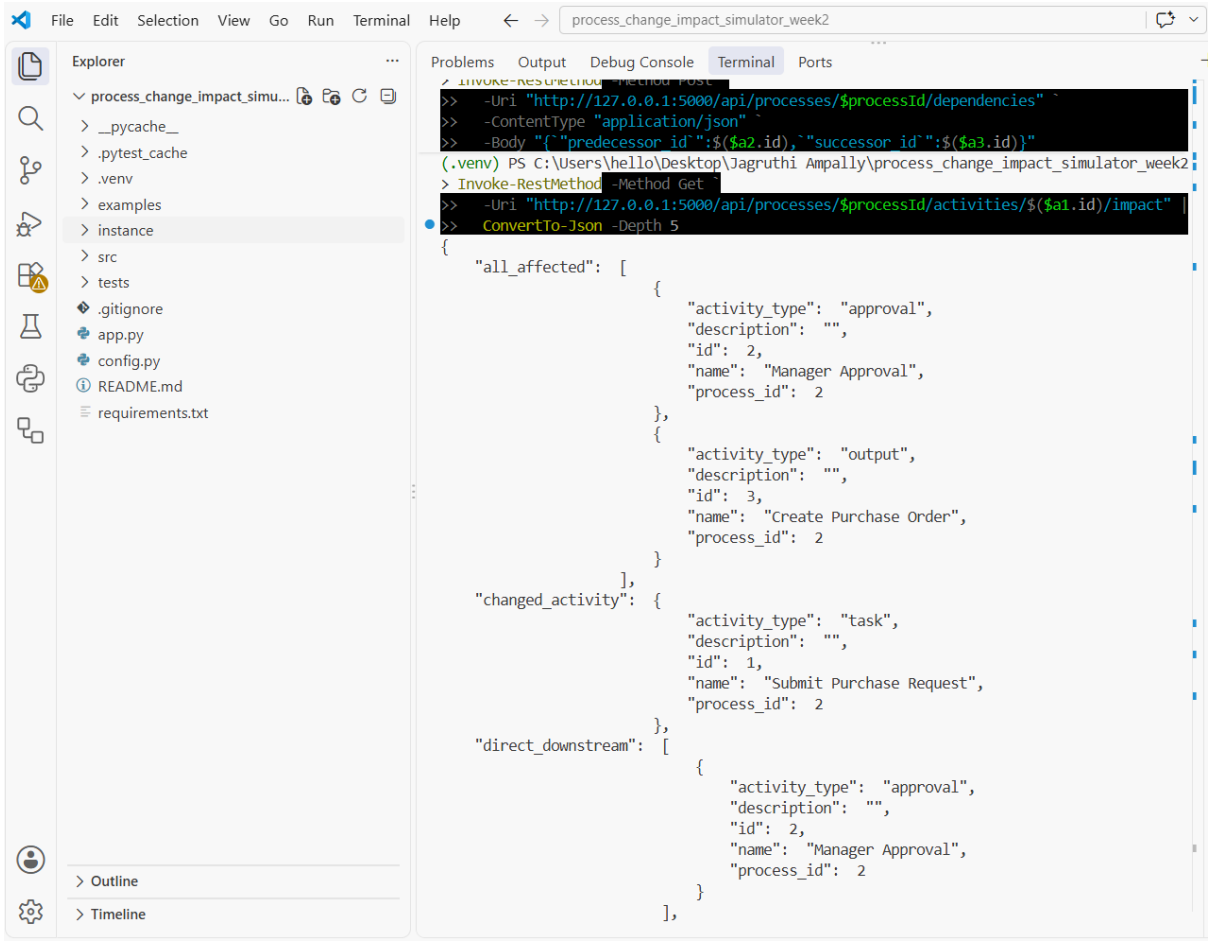
| Source file or component | Responsibility |
|---|---|
| app.py | Creates the Flask application, loads configuration, initializes the database, registers the API routes, and provides the home endpoint. |
| config.py | Stores the database path and application configuration. |
| src/process_simulator/database.py | Opens SQLite connections, enables foreign-key checks, and initializes the database using the SQL schema. |
| src/process_simulator/schema.sql | Defines the processes, activities, and dependencies tables and their relationships. |
| src/process_simulator/repository.py | Contains database operations for creating and retrieving processes, activities, and dependencies. |
| src/process_simulator/routes.py | Defines the API endpoints for process creation and listing, activity creation, dependency creation, JSON import, and impact analysis. |
| src/process_simulator/importer.py | Checks imported JSON data, validates activity keys and dependency references, detects cycles, and saves valid imports. |
| src/process_simulator/graph.py | Builds the directed graph, checks for cycles, and calculates direct, indirect, and total downstream impact. |
| requirements.txt | Lists Flask, NetworkX, and pytest as project dependencies. |
| examples/sample_process.json | Provides an example process definition that can be used to test JSON import. |
| tests/conftest.py | Creates a test Flask application that uses a temporary SQLite database. |
| tests/test_core.py | Contains automated tests for process operations, dependency analysis, JSON import, validation, and cycle rejection. |

## 3. Tests Performed and Results

The project was run locally on Windows using Python 3.13.0 in a virtual environment. The Flask application started successfully and returned a JSON response at http://127.0.0.1:5000 showing the application name and running status.




Explorer
process_change_impact_simu...
__pycache__
.pytest_cache
.venv
examples
instance
src
tests
.gitignore
app.py
config.py
README.md
requirements.txt
Problems
Output
Debug Console
Terminal
Ports
(.venv) PS C:\Users\hello\Desktop\Jagruthi Ampally\process_change_impact_simulator_week2
> $a1 = Invoke-RestMethod -Method Post `
>> -Uri "http://127.0.0.1:5000/api/processes/$processId/activities" `
>> -ContentType "application/json" `
>> -Body '{"name":"Submit Purchase Request","activity_type":"task"}'
>> $a1
>> $a2
>> $a3
activity_type : task
id : 1
name : Submit Purchase Request
process_id : 2
activity_type : approval
description :
id : 2
name : Manager Approval
process_id : 2
description :
id : 3
name : Create Purchase Order
process_id : 2
(.venv) PS C:\Users\hello\Desktop\Jagruthi Ampally\process_change_impact_simulator_week2
> Invoke-RestMethod -Method Post `
>> -ContentType "application/json" `
>> -Body "{`"predecessor_id`":$($a1.id),`"successor_id`":$($a2.id)}"
dependency_id status
1 created


File
Edit
Selection
View
Go
Run
Terminal
Help
process_change_impact_simulator_week2
Explorer
process_change_impact_simu...
__pycache__
.pytest_cache
.venv
examples
instance
src
tests
.gitignore
app.py
config.py
README.md
requirements.txt
Outline
Timeline
Problems
Output
Debug Console
Terminal
Ports
>> -Uri "http://127.0.0.1:5000/api/processes/$processId/dependencies" `
>> -ContentType "application/json" `
>> -Body "{`"predecessor_id`":$($a2.id),`"successor_id`":$($a3.id)}"
(.venv) PS C:\Users\hello\Desktop\Jagruthi Ampally\process_change_impact_simulator_week2
> Invoke-RestMethod -Method Get `
>> -Uri "http://127.0.0.1:5000/api/processes/$processId/activities/$($a1.id)/impact" |
>> ConvertTo-Json -Depth 5
{
"all_affected": [
{
"activity_type": "approval",
"description": "",
"id": 2,
"name": "Manager Approval",
"process_id": 2
},
{
"activity_type": "output",
"description": "",
"id": 3,
"name": "Create Purchase Order",
"process_id": 2
}
],
"changed_activity": {
"activity_type": "task",
"description": "",
"id": 1,
"name": "Submit Purchase Request",
"process_id": 2
},
"direct_downstream": [
{
"activity_type": "approval",
"description": "",
"id": 2,
"name": "Manager Approval",
"process_id": 2
}
],

The automated tests were run with the command `python -m pytest -q`. The terminal output was `4 passed in 0.47s`. All four tests passed.

| Test performed | Observed result |
|---|---|
| Process creation and listing; activity and dependency creation | Passed |
| Downstream impact: direct and indirect activities | Passed |
| Valid JSON import and rejection of a missing dependency reference | Passed |
| Rejection of an imported process containing a circular dependency | Passed |

The process API was also tested manually using PowerShell. A process named Purchase Approval was created with process ID 2. Three activities were added: Submit Purchase Request (ID 1), Manager Approval (ID 2), and Create Purchase Order (ID 3). Two dependencies were created: activity 1 to activity 2, and activity 2 to activity 3. The impact analysis response correctly listed Manager Approval as the direct downstream activity and Create Purchase Order as the indirect downstream activity. Both activities appeared in the all_affected list.

The circular-dependency rejection was verified by the automated JSON-import test. A separate manual API test for adding a cycle was not included in the terminal results shared for this document.

## 4. Technical Problems Encountered

- **Python version selection:** The first virtual-environment command requested Python 3.12, but that version was not installed. The computer had Python 3.13.0, 3.11, and 3.10. The environment was created successfully with `py -3.13 -m venv .venv`.
- **PowerShell activation policy:** PowerShell initially required an execution-policy command to activate the virtual environment. The process-level RemoteSigned setting allowed activation for the current terminal session.
- **Pytest could not find app.py:** The first `pytest -q` command reported `ModuleNotFoundError: No module named 'app'` while loading conftest.py. Running the test suite as `python -m pytest -q` from the project root resolved the import issue, and all four tests passed.
- **Reserved PowerShell variable:** The variable name `$pid` could not be assigned because PowerShell uses `$PID` as a built-in read-only variable. Changing the variable name to `$processId` allowed the API commands to run.
- **Missing process response after the variable error:** The activity requests initially returned 'Process not found' because the process ID variable had not been set. Re-running the process creation and using `$processId` resolved this issue.

## 5. Functionality Still Incomplete

The Week 2 implementation is an API-focused foundation. The following planned requirements and features have not yet been implemented or verified in this version:

- Process editing and deletion, including FR-04, FR-06, and FR-07.
- Starting and ending activity management (FR-11).
- Exporting process definitions to JSON (FR-16).
- Editing and removing dependencies (FR-10).
- Change scenario creation, editing, deletion, and storage (FR-19 to FR-25).
- Simulation of original and modified process paths, including condition handling and comparison (FR-33 to FR-40).
- Visual process graph display and highlighting of affected activities (FR-41 to FR-43).
- Manager-facing result screens and report export (FR-44 to FR-46).
- Administration screens, application error review, and related administration functions (FR-47 to FR-50).
- A complete browser-based user interface for creating and managing processes. The current operations are provided through Flask JSON API endpoints.
- Additional tests for more input cases, database integrity, error handling, and the remaining planned requirements.

## 6. Conclusion

The Week 2 work demonstrates more than application startup. The application can save a process, add activities, connect activities, validate and import process JSON, and identify direct and indirect downstream activities. The automated test run completed with four passing tests, and the purchase approval workflow was also exercised through the API. The next development work will build on this foundation by adding change scenarios, simulation comparisons, a visual interface, and reporting.